

1 把公式落实到现有 Python 项目

这一节按实际计算顺序阅读现有代码。先理解一个数组代表什么，再看如何更新状态，最后看误差怎样汇总成报告。这里的短代码取自原程序或本讲义附带的教学脚本；为便于阅读而省略上下文的片段不应单独当作完整程序运行。原项目解决大规模实验与报告生成，新增的 `teaching_demo.py` 则保留学习所需的核心计算，只依赖 NumPy，默认打印结果，不写文件。

1.1 文件之间怎样配合

文件	职责与阅读重点
<code>gbm.py</code>	生成共同布朗增量；计算精确 GBM、价格 EM、Milstein；独立预试验拟合控制变量；保存收敛统计、终值样本和图。
<code>nonaffine.py</code>	生成相关布朗增量；对 (X, Y) 做 log-EM；比较粗网格与三层参考网格；记录状态与参考敏感性。
<code>verify.py</code>	用确定性输入和另一种写法核对算法，再由保存的终值数组重新算误差，检查统计是否一致。
<code>build_report_data.py</code>	从已有 JSON/CSV 生成报告使用的数值宏、八张表和四幅图，并更新 <code>numerical_summary.txt</code> 。
<code>run_all.py</code>	依次运行上面四个脚本；某一步失败就停止；它本身不编译 L ^A T _E X。
<code>README.md</code>	记录依赖、实验规模、命令、误差定义和解释边界。

`results/` 保存计算结果，`figures/` 保存图，原报告的 `.tex` 把这些结果排版。因而“重画已有结果”“重新模拟所有路径”“重新编译 PDF”是三个不同操作。原项目的这几个生成脚本使用固定输出文件名；学习时先运行新增教学脚本，可以保留现有实验结果。

1.2 NumPy 数组：先问每一个轴代表什么

NumPy 的 `array` 是规则排列的数。`shape` 给出每个轴上的长度，轴编号从 0 开始。设本批有 B 条独立路径，最细网格有 N_f 步。GBM 使用“路径在前”的数组：

```
1 fine = rng.normal(0, math.sqrt(T / nfine), (size, nfine))
2 w = fine.sum(axis=1)
```

这里 `size` 就是 B 。`fine.shape == (B, Nf)`，`fine[j,k]` 是第 j 条路径、第 k 个小时时间段的布朗增量。每一行是一条路径。正态生成器的第二个参数是标准差，所以传入 $\sqrt{T/N_f}$ ，不是方差 T/N_f 。沿 `axis=1` 求和，就是把每一行的所有时间增量相加，得到 W_T ；结果 `w.shape == (B,)`。

若粗网格有 N 步，且 $r = N_f/N$ 是整数，原代码写作

```
1 dw = fine.reshape(size, int(n), nfine // int(n)).sum(axis=2)
```

`reshape` 不重新抽随机数，它把原有顺序整理为 (B, N, r) 。最后一个轴含有同一粗区间内连续的 r 个细增量；沿该轴求和，输出 (B, N) 。例如单条路径八个细增量为

$$(a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8),$$

取 $N = 2$ 时，先分为 $(a_1, \dots, a_4)$ 与 $(a_5, \dots, a_8)$ ，再变为 $(a_1 + \dots + a_4, a_5 + \dots + a_8)$ 。这是“粗细路径共用同一布朗运动”的代码实现。若另抽一套粗增量，即使分布正确，逐路径的差也不再衡量同一随机驱动下的离散误差。

非仿射程序则使用“时间在前”的数组。两种布局都正确，但求和轴不同：

操作	GBM	非仿射模型
最细增量形状	(B, N_f)	$(N_f, B, 2)$
一个元素	第 j 条路径、第 k 步	第 k 步、第 j 条路径、第 q 个噪声
粗化前重新分组	(B, N, r)	$(N, r, B, 2)$
应求和的轴	<code>axis=2</code>	<code>axis=1</code>
粗化后形状	(B, N)	$(N, B, 2)$

不要只背 `axis=1` 或 `axis=2`；应先写出形状，再指认“要合并的细时间轴”。类似地，`sum(axis=0)` 在 $(B, 2)$ 上是跨路径汇总，结果中保留两种方法。

1.3 GBM：从递推式到 `prod`

因为 GBM 的系数都正比于价格，价格 EM 和 Milstein 都能写为“旧价格乘一个因子”。原 `simulate_batch` 的核心是：

```

1 exact = S0 * np.exp((MU - 0.5 * SIGMA**2) * T + SIGMA * w)
2 dw2 = dw * dw
3 base = 1 + MU * h + SIGMA * dw
4 em = S0 * np.prod(base, axis=1)
5 mil = S0 * np.prod(
6     base + 0.5 * SIGMA**2 * (dw2 - h), axis=1)
7 endpoints = np.column_stack((em, mil))

```

`dw*dw` 是每个位置分别平方，`**2` 也是平方；它们不是矩阵乘法。`base` 的形状为 (B, N) ，`prod(axis=1)` 把每条路径上 N 个时间因子相乘，于是一次得到所有终值。数学上，这只是

$$S_N = S_0 F_0 F_1 \cdots F_{N-1}$$

的简写，没有改变递推格式。若要画整条路径，需要 `cumprod` 保存逐步累积乘积；终值误差实验只需 `prod`。精确路径图类似地用 `cumsum` 得到各时刻的布朗运动。

`column_stack` 把两个长度为 B 的数组并成两列，所以 `endpoints.shape == (B, 2)`，第 0 列为 EM，第 1 列为 Milstein。精确终值 `exact` 的形状却是 $(B,)$ 。作差时要写

```

1 error = endpoints - exact[:, None]

```

：表示取全部路径，`None` 插入一个新轴，令形状从 $(B,)$ 变为 $(B, 1)$ 。NumPy 的广播机制把这一列用于两列数值解，得到 $(B, 2)$ 的配对误差。它没有给每种方法重新生成一条精确路径。例如两条路径有

$$\text{endpoints} = \begin{pmatrix} 99 & 100 \\ 110 & 112 \end{pmatrix}, \quad \text{exact[:, None]} = \begin{pmatrix} 101 \\ 111 \end{pmatrix}, \quad \text{error} = \begin{pmatrix} -2 & -1 \\ -1 & 1 \end{pmatrix}.$$

这里的减法在“相同路径”内完成，这是配对估计。

`simulate_batch` 中的 `yield` 使函数成为生成器。调用者用 `for` 逐层取出四项：步数 N 、精确终值、两列数值终值、控制变量。程序不会一次保存每个网格的所有中间状态；所有网格共享本批已经生成的 `fine`。

1.4 强误差、弱偏差、控制变量各占哪一列

原程序没有把全部误差样本永久保存在内存，而是对三个指标累积总和与平方和：

```
1 corrected = error - controls @ betas[k]
2 metrics = np.stack((np.abs(error), error, corrected), axis=2)
3 sums[k] += metrics.sum(axis=0)
4 squares[k] += (metrics * metrics).sum(axis=0)
```

此处 `metrics.shape == (B,2,3)`：第一个轴是路径，第二个轴是方法，第三个轴依次是绝对终值误差、带符号配对差、控制变量修正后的带符号差。跨路径求和后形状为 $(2,3)$ 。外层 `sums.shape == (L,2,3)`，其中 L 是网格层数， k 指当前网格。

三列对应的总体目标依次是 $\mathbb{E}|S_T^h - S_T|$ 、 $\mathbb{E}[S_T^h - S_T]$ 、以及同一个弱偏差。后两列目标相同，估计噪声不同。不能把 `np.abs(error)` 当成弱误差，也不能先将弱样本逐个取绝对值再平均。

在某一网格上，`controls.shape == (B,5)`，`betas[k].shape == (5,2)`。符号 `@` 表示矩阵乘法，故乘积是 $(B,2)$ ，为每条路径、每种方法分别计算五个控制变量的加权和。控制变量的严格零均值及方差缩减原理在理论部分已说明；对应代码的关键是：系数只在独立预试验中拟合，然后在正式样本中固定使用。

`fit_controls` 分批记录 X 与 Y 的总和、 $X^T X$ 和 $X^T Y$ 。 X 的五列是控制变量， Y 的两列是 EM/Milstein 配对误差。总预试验样本数记为 M_0 ，最终进行中心化最小二乘：

```
1 covx = sxx[k] - np.outer(sx[k], sx[k]) / count
2 covxy = sxy[k] - np.outer(sx[k], sy[k]) / count
3 betas[k] = np.linalg.solve(covx, covxy)
```

`outer` 是外积；例如两个长度分别为 5 和 2 的向量，外积为 5×2 矩阵。这里 `covx` 与 `covxy` 是尚未除以 $M_0 - 1$ 的中心化交叉乘积；同一个分母会在最小二乘方程两边抵消。`solve(A,B)` 求解 $AZ = B$ ，不必显式计算矩阵逆。预试验点数过少会使拟合不稳定，不能随意用几条路径代替原来的 20,000 条预试验路径。

原代码分别使用 `default_rng(SEED+1)` 和 `default_rng(SEED)` 建立预试验与正式实验的随机流。拟合阶段的中心化用于估计协方差；正式校正时使用理论已知为零均值的控制变量本身，不是用正式样本估计一个均值后再从每个控制变量中扣除。

1.5 批量累计：平均数和标准误怎样得到

如果 $V_1, \dots, V_M$ 是不同路径上的某个指标，只需保存

$$M, \quad A = \sum_{j=1}^M V_j, \quad B_2 = \sum_{j=1}^M V_j^2,$$

就能计算

$$\bar{V} = \frac{A}{M}, \quad s^2 = \frac{B_2 - A^2/M}{M-1}, \quad \text{SE}(\bar{V}) = \frac{s}{\sqrt{M}}. \quad (1.1)$$

`gbm.py` 的 `mean_se(total,squares,count)` 把这三个公式一次应用到所有网格、方法、指标。`np.maximum(...,0)` 避免理论上非负的方差因浮点舍入产生很小的负数；它不能修复错误的数据。

非仿射程序把同一逻辑封装进 `Moments` 对象，其接口是：

```

1 strong[n, r].add(np.abs(terminal[n] - terminal[r]))
2 weak[n, r].add(payoff[n] - payoff[r])
3 result = strong[n, r].summary()

```

`add(values)` 接收一批独立路径的观测值，更新样本数、总和与平方和；`summary()` 返回均值、标准误、95% 区间上下端点等字段。上式字典的键 (n, r) 表示“粗网格 n 步，对照参考网格 r 步”。元组键不是乘法。

这样分批的统计结果与把所有样本放在一起平均等价，只有浮点加法顺序带来的微小差别。分批还能限制布朗增量数组的内存占用。不过原项目为绘图和事后检查，另外保留了部分或全部终值样本；它并不是完全不存样本。若数据均值极大、方差极小，式 (1.1) 的两大数相减会损失精度，可改用 Welford 算法；现有程序采用的是总和与平方和形式。

跨网格数据相关，因为它们共用布朗路径；同一网格上不同路径仍是 Monte Carlo 平均所用的独立观测。因此逐点标准误有明确意义，但各层 95% 区间不是“所有层同时覆盖真值的 95% 区间”。

1.6 非仿射模型：相关噪声和同时更新

`nonaffine.py` 首先生成两个独立标准正态数组，然后在线性组合中加入相关性：

```

1 dw = rng.standard_normal((finest, m, 2))
2 dw[:, :, 1] = (
3     PARAMETERS["rho"] * dw[:, :, 0]
4     + math.sqrt(1 - PARAMETERS["rho"]**2) * dw[:, :, 1])
5 dw *= math.sqrt(hfinest)

```

赋值右侧计算时，`dw[:, :, 1]` 还是原先独立的第二个正态噪声。令 Z_1, Z_2 独立标准正态，则最终的两个增量为

$$\Delta W_1 = \sqrt{h}Z_1, \quad \Delta W_2 = \sqrt{h}(\rho Z_1 + \sqrt{1 - \rho^2}Z_2).$$

直接用概率论可核对：两者方差均为 h ，协方差为 ρh ，相关系数为 ρ 。这里的第三个轴长度为 2，记录两种噪声；“路径条数很多”不等于“每条路径有很多独立噪声分量”。

`em_terminal` 将最细网格增量按时间分组：

```

1 finest, n_paths, _ = dw.shape
2 ratio = finest // n_steps
3 increments = dw if ratio == 1 else dw.reshape(
4     n_steps, ratio, n_paths, 2).sum(axis=1)

```

如果本层已经是最细网格，直接复用原数组；否则将相邻细增量相加。原实验的所有步数都整除 8192；教学脚本另外显式检查整除关系，避免误用不相容的网格。

在计算内核里， x 保存 $X = \log S$ ， y 保存 Y 。对每个时刻、每条路径，原程序用旧状态计算两个新状态：

```

1 old_y = y[j]
2 g = g_min + (g_max - g_min) * sigmoid
3 x[j] += (mu - 0.5*g*g)*h + g*increments[step, j, 0]
4 y[j] += (kappa*(theta-old_y)*h
5     + xi*math.sqrt(1.0+old_y*old_y)*increments[step, j, 1])

```

这里 `sigmoid` 也是在 `old_y` 上计算的。`+=` 表示将右侧增量加到左侧旧值。代码先写 X 更新、再写 Y 更新，并不意味着第二个公式使用了新 Y ；两个公式的系数都在 Y_n 上取值。这正是“同时更新”的含义。若先改写 Y 、再用新 Y 计算波动率，就已经换成了另一种离散方法。

教学脚本对全部路径一次进行向量运算，写 `old_y = y.copy()` 后再更新。这里需要 `copy()` 的原因是：`old_y = y` 只让两个变量名指向同一个数组；后续原地修改 `y` 会同时改变它。原 Numba 内核里的 `old_y = y[j]` 是标量读取，不存在这个数组别名问题。

逻辑函数 $\ell(y) = 1/(1 + e^{-y})$ 的稳定计算也值得读懂：

```
1 if old_y >= 0:
2     sigmoid = 1.0 / (1.0 + math.exp(-old_y))
3 else:
4     exp_y = math.exp(old_y)
5     sigmoid = exp_y / (1.0 + exp_y)
```

$y \geq 0$ 时 $-y \leq 0$ ， $y < 0$ 时直接算 e^y ，所以两分支都不会计算极大的正指数。两式数学上相等，但这样避免 y 很负时 e^{-y} 溢出。教学脚本用布尔掩码实现同样的两分支；不直接用可能同时计算两边的 `np.where`。最终才用 `np.exp(x)` 还原价格，并检查其有限性与正性。理论上的 $e^x > 0$ 不会自动阻止计算机上的上溢或下溢，因此原项目也记录中间 X, Y 的范围和状态诊断。

1.7 为什么原非仿射代码使用 Numba

原非仿射实验包含 100,000 条路径，以及 $N = 8, 16, 32, 64, 128, 256, 2048, 4096, 8192$ 共九层网格。总路径步更新数为

$$100000(8 + 16 + 32 + 64 + 128 + 256 + 2048 + 4096 + 8192) = 1\,484\,000\,000.$$

纯 Python 的双重逐项循环开销较大。`@njit(cache=False)` 让 Numba 在首次调用时把内核编译为较快的机器代码；公式没有因编译而改变。`run` 先用一个很小的零增量输入预热编译，并单独记录编译耗时，再开始统计主实验时间。因此不能只比较两个脚本日志里的秒数，就宣称某种数学方法更快：样本数、网格、绘图和预试验范围本来不同。

教学脚本去掉 Numba，时间步仍循环，但在每一步使用 NumPy 同时更新整批路径。它的较小网格足以学习算法，也能在本机当前 Python/NumPy 环境中直接运行。它不会冒充原来十万路径、最细 8192 步的大规模实验。

1.8 结果文件、日志与报告数字怎样连接

CSV 是带列名的表格，例如 `gbm_convergence.csv` 每行对应一种方法和一个网格，并分别保存强误差、原始弱偏差、控制变量弱偏差及其标准误。JSON 适合嵌套结构：参数、随机种子、软件版本、拟合窗口、诊断和结果可以放进同一份摘要。NPZ 是 NumPy 的压缩数组容器，保存真实终值样本，便于事后从样本重新计算统计。

例如原非仿射结果存为 `nonaffine_terminals.npz`，其中键 `S_64` 表示 64 步网格的全部终值，`S_8192` 表示最细参考的全部终值。下面是一段只读练习，须在项目根目录运行：

```
1 from pathlib import Path
2 import numpy as np
3
4 with np.load(Path("results") / "nonaffine_terminals.npz") as data:
```

```

5 coarse = data["S_64"]
6 reference = data["S_8192"]
7 strong = np.mean(np.abs(coarse - reference))
8 difference = (np.maximum(coarse - 100.0, 0.0)
9               - np.maximum(reference - 100.0, 0.0))
10 weak = difference.mean()
11 se = difference.std(ddof=1) / np.sqrt(difference.size)
12 print(strong, weak, se)

```

`maximum(s-100,0)` 是逐元素计算看涨收益。`ddof=1` 表示样本方差用分母 $M - 1$ 。标准误必须从配对差 `difference` 计算，不能忽略粗细样本的协方差。

`verify.py` 就包括这种保存结果的交叉检查，还用预先指定的正弦、余弦小数数组比较“向量化写法”和“逐条递推写法”。这些小数数组不是布朗运动样本；它们的用途是检验代数实现，不是检验随机模型的分布。它还检查常波动率情况下 `log-EM` 是否退化为精确 `GBM`。通过后写入 `results/verification.json`。该脚本会导入 `nonaffine.py`，因此也需要 `Numba`；它不会重新模拟全部路径，但不是完全不写文件的检查。

`build_report_data.py` 不再抽样，而是读 `JSON/CSV`，生成 `report_values.tex` 与各表格的 `.tex`，并更新报告图。这让报告中“`EM 强阶`”等数字直接来自保存结果。`np.polyfit(log(h),log(error),1)` 返回拟合直线的斜率与截距，斜率对应经验幂指数。程序对非仿射弱差先检查选定窗口中的区间是否排除零；若有区间含零，`rate_fit` 返回 `slope=None`。即使均排除零，原程序仍将有限参考下的斜率标为描述性结果。

`GBM` 的 `log` 函数一边打印、一边把内容加入列表，最后写入 `gbm_run.log`。非仿射程序打印进度和摘要，并把时间、种子与环境写进 `JSON`；`run_all.py` 本身没有把所有终端输出自动集成总日志。需要记录个人演示输出时，可明确重定向到讲义目录中的新文件。

1.9 从小练习开始，再决定是否完整复现实验

本讲义新增 `tutorial_zh/teaching_demo.py`。在项目根目录打开终端，依次运行：

```

1 python3 tutorial_zh/teaching_demo.py --self-check
2 python3 tutorial_zh/teaching_demo.py
3 python3 tutorial_zh/teaching_demo.py --paths 8000 --seed 2026090804

```

第一条只执行确定性检查，包括 `GBM` 递推与精确公式、粗化后总增量一致、非仿射旧状态同时更新、常波动率退化、极端输入下逻辑函数稳定性，以及分批统计的标准误。第二条默认使用 2000 条路径、批大小 256；`GBM` 的层数为 8, 16, ..., 512，非仿射粗网格为 16, 32, 64, 128，参考网格为 256, 512, 1024。第三条增加路径数，适合观察区间通常如何缩窄。脚本没有控制变量，也不自动拟合或认证收敛阶。

本次在本机 `Python 3.9.6`、`NumPy 2.0.2` 上，确定性检查通过，默认演示运行通过。例如 `GBM` 的 $N = 512$ 时，`EM 强误差` 约为 0.236916，`Milstein` 约为 0.003028；`EM 的原始弱差` 约为 -0.001314，其 95% 区间约为 $[-0.014883, 0.012255]$ ，而解析均值偏差约为 -0.000257。这个小练习同时展示了“强误差明显可见”和“弱偏差可能被抽样噪声淹没”。这些是教学脚本自己的结果，不替换原报告的统计数字。

非仿射演示的参考只到 1024 步，默认样本中 $512 \rightarrow 1024$ 的平均绝对终值差约为 0.116254。与某个粗网格差相比，它可能并不小；这正好提醒我们检查参考误差，不能把最细曲线直接命名

为精确解。更换种子后个别区间或误差大小会变化，不应要求每次结果都整齐地单调下降。

原项目完整运行使用 README 中记录的依赖环境。其 `requirements.txt` 固定 NumPy、Matplotlib 和 Numba 版本；已有实验元数据来自另一套环境，因此本机能运行 NumPy 教学版，并不表示已复现原环境。准备好兼容依赖、并在项目的副本中操作时，完整实验命令为：

```
1 python3 run_all.py
```

或者分别执行：

```
1 python3 gbm.py --paths 160000 --pilot-paths 20000 --batch-size 2000
2 python3 nonaffine.py --paths 100000 --batch-size 512 --seed 2026090804
3 python3 verify.py
4 python3 build_report_data.py
```

原 GBM 的种子写在常量 `SEED` 中，没有 `--seed` 命令行选项。非仿射与教学脚本有该选项。原报告源是英文，可按其 README 用 `pdflatex` 编译；本中文讲义采用支持中文的引擎，编译方法见随讲义生成的说明。

最后要区分“同分布复现实验”和“尽量得到同一串输出数字”。固定种子决定伪随机流，但还应保留生成器类型、软件版本、路径数、网格、批大小及抽样调用顺序。非仿射数组是 $(N_f, B, 2)$ ；改变 B 后，同一串随机数会分配到不同的路径和时间位置，所以即使种子相同，逐路径终值仍会改变。GBM 与非仿射在原项目中使用相同种子，也不意味着二者共享可逐路径比较的噪声；它们是两个不同实验。教学脚本有意分别使用传入的种子及其加一值，并将它们打印出来。

建议实际练习时先读清一批的数组形状，再手算一条两步路径，接着核对默认输出中的强误差与带符号弱差，最后查看保存终值能否还原原报告统计。这样每一层代码都有可解释、可核对的数学对象。